

NanoPoreSeq

NanoPoreSeq — Rapport de relecture, procédures et corrections

---

Simulateur de BAM Nanopore de tumeurs du SNC pour tester Rapid-CNS2_nf

Préparé pour Pierre Bady · 26 octobre 2026 · 12 scripts relus en cinq passes

Ce projet est développé avec l'aide de l'IA (Claude, Anthropic), sous la relecture active de Pierre Bady.

Plan du document

1. Synthèse
2. Organisation des fichiers
3. Installation et prérequis
4. Procédures d'utilisation
5. Fonctionnement détaillé
6. Description des scripts et de leurs fonctions
7. Erreurs trouvées et corrections
8. Explication des choix
9. Tests réalisés
10. Limites et recommandations

1. Synthèse

Les 12 scripts du projet ont été relus ligne à ligne en cinq passes, corrigés, puis regroupés en trois dossiers selon leur usage : simulation, lancement de Rapid-CNS2 et clé USB autonome. Chaque correction a été vérifiée par un test exécuté sur des BAM simulés ; les scripts R ont été exécutés sur les données réelles fournies avec Rapid-CNS2 (Capper et al.).

L'erreur la plus importante concernait bedmethyl_from_bam.py : les appels de méthylation des reads du brin – étaient placés à des positions fausses (95 % hors CpG), si bien que la moitié des données transmises à la classification était du bruit. Deux autres défauts avaient un impact pratique : la consommation mémoire du simulateur (un individu de cohorte tué par manque de mémoire) et une couverture MGMT trop faible (~4–7×) pour le seuil de 10× utilisé par les lanceurs.

Après correction, les mesures sur BAM simulés retrouvent les valeurs attendues (méthylation MGMT 0,89 pour 0,88 attendu ; sondes HM450 transplantées 0,03 / 0,91 pour 0,04 / 0,92 ; codélétion 19q 0,50 ; couverture MGMT 18–22×), la mémoire de pointe est divisée par 2,7, une cohorte test de 3 individus est générée sans échec et la classification R d'un BAM gbm simulé donne la classe attendue, GBM_RTK_II.

Ce rapport décrit ensuite, pas à pas, les procédures d'installation et d'utilisation (section 4), le fonctionnement interne de chaque étape (section 5), les fonctions de chaque script (section 6), puis les corrections, les choix, les tests et les limites.

2. Organisation des fichiers

Les scripts sont regroupés par usage. À l'intérieur d'un dossier, les scripts s'appellent par leur chemin relatif : il faut garder ensemble les fichiers d'un même dossier.

Dossier	Usage	Fichiers
`simulation/`	Simuler des BAM, les vérifier, simuler des cohortes et classer leur méthylation	`simulate_glioma_hg38.py, verify_bam.py, build_cohort.py, bedmethyl_from_bam.py, methylation_classification_local.R, 01_export_probe_targets.R, 02_export_cohort_probe_targets.R, run_full_mnp_classification.sh, simnanocns_gui_en.py`
`rapidcns2/`	Lancer le pipeline Rapid-CNS2_nf ; préparer des données réelles	`rapidcns2_gui_en.py, align_fastq_rapidcns2_en.sh`
`usb/`	Construire une clé USB autonome (pipeline, conteneurs, références, interface)	`make_usb_stick_en.sh, prepare_usb_en.sh`
`docs/`	Guides utilisateur et rapports de relecture	`NanoPoreSeq_guide_utilisateur_FR.docx, NanoPoreSeq_user_guide_EN.docx, NanoPoreSeq_rapport_FR.docx, NanoPoreSeq_report_EN.docx`

Arborescence du projet

```
NanoPoreSeq/
├── README.md                                  présentation des dossiers
├── simulation/                                simuler, vérifier, classer
│   ├── simulate_glioma_hg38.py                simule un BAM
│   ├── verify_bam.py                          vérifie un BAM
│   ├── build_cohort.py                        simule une cohorte
│   ├── bedmethyl_from_bam.py                  BAM → bedMethyl
│   ├── methylation_classification_local.R     classe la méthylation
│   ├── 01_export_probe_targets.R              1 profil réel par classe
│   ├── 02_export_cohort_probe_targets.R       N profils réels par classe
│   ├── run_full_mnp_classification.sh         chaîne complète
│   └── simnanocns_gui_en.py                   interface Streamlit
├── rapidcns2/                                 lancer Rapid-CNS2_nf
│   ├── rapidcns2_gui_en.py                    interface Tkinter
│   └── align_fastq_rapidcns2_en.sh            FASTQ → BAM (données réelles)
├── usb/
│   ├── make_usb_stick_en.sh                   formate et crée la clé USB (tout-en-un)
│   └── prepare_usb_en.sh                      copie le contenu sur la clé
└── docs/                                      guides et rapports (FR, EN ; .docx et .pdf)
```

Les arborescences des dossiers produits par les scripts (échantillon, cohorte, classification, résultats de Rapid-CNS2, clé USB) sont décrites dans le guide utilisateur, section 8.5.

Enchaînement général

1. Simulation : simulate_glioma_hg38.py (un échantillon) ou build_cohort.py (N échantillons) produit des BAM hg38 avec tags MM/ML et un fichier de vérité terrain.
2. Contrôle : verify_bam.py compare le BAM à la vérité terrain.
3. Classification locale (facultatif) : bedmethyl_from_bam.py puis methylation_classification_local.R donnent la classe de méthylation attendue par Rapid-CNS2.
4. Test du pipeline : rapidcns2_gui_en.py (ou la clé USB) lance Rapid-CNS2_nf sur les BAM ; le rapport produit est comparé à la vérité terrain.

3. Installation et prérequis

3.1 Logiciels

1. Python 3 avec pysam et numpy (simulation, vérification, bedMethyl).

   ```
   python3 -m pip install pysam numpy
   ```

2. Interface de simulation : Streamlit (version 1.37 ou plus) et pandas.

   ```
   python3 -m pip install "streamlit>=1.37" pandas
   ```

3. Interface Rapid-CNS2 : tkinter.

   ```
   sudo apt install python3-tk
   ```

4. Classification : R avec les paquets optparse, ranger, matrixStats, data.table, glmnet et GenomicRanges (Bioconductor).

   ```
   Rscript -e
   'install.packages(c("optparse","ranger","matrixStats","data.table","glmnet","BiocManager")
   )'
   Rscript -e 'BiocManager::install("GenomicRanges")'
   # ou, sous Ubuntu (méthode utilisée pour les tests) :
   sudo apt install r-base-core r-cran-optparse r-cran-ranger r-cran-matrixstats \
       r-cran-data.table r-cran-glmnet r-bioc-genomicranges
   ```

5. Pipeline : Nextflow, Java 17 ou plus, Docker ou Apptainer/Singularity. Données réelles : dorado et samtools. Clé USB : rsync et curl.

3.2 Données

1. Le dossier data/ de Rapid-CNS2_nf, qui fournit le panel, les régions et les données d'entraînement.

```
git clone https://github.com/areebapatel/Rapid-CNS2_nf
export DATA=$PWD/Rapid-CNS2_nf/data
```

2. Fichiers utilisés : NPHD_panel_hg38.bed (panel), mgmt_hg38.bed (promoteur MGMT), genes.bed (EGFR, CDKN2A/B), refGene.txt.gz (sites ATRX et CIC) ; pour la classification : top_probes_hm450.Rdata, HM450.hg38.manifest.gencode.v22.Rdata, capper_top_100k_betas_binarised.Rdata.
3. Une FASTA hg38 dont les contigs s'appellent chr1… (UCSC) ou NC_000001.11… (NCBI RefSeq). Son index .fai est créé automatiquement au premier usage si le dossier est accessible en écriture.

```
export REF=/export/scratch3/data/Annotations/UCSC_HG38/hg38.fa
```

3.3 Contrôle de l'installation

1. Les scripts en ligne de commande affichent leur aide avec --help ; un premier test rapide sur un seul chromosome avec la séquence de remplacement (--standin) vérifie la chaîne sans référence réelle (moins d'une minute pour chr10 ou chr17 dans nos tests).

```
cd NanoPoreSeq/simulation
python3 simulate_glioma_hg38.py --help
python3 simulate_glioma_hg38.py --profile gbm --standin --data-dir $DATA --out test_gbm --
chroms chr10
```

4. Procédures d'utilisation

Chaque procédure indique son objectif, les étapes avec les commandes, les fichiers produits et les points à contrôler. Les commandes supposent que les variables DATA et REF sont définies (section 3) et qu'on se trouve dans le dossier indiqué.

4.1 Simuler un échantillon et le vérifier

Objectif : produire un BAM d'un profil donné (gbm, astro ou oligo), puis vérifier qu'il contient bien les altérations simulées. `[simulation/]`

Étapes

1. Choisir le profil et la pureté tumorale. La VAF des variants hétérozygotes vaut pureté/2 ; TERT C228T du gbm reste fixé à 0,80.
2. Faire un test rapide sur un ou deux chromosomes avec la séquence de remplacement.

```
python3 simulate_glioma_hg38.py --profile oligo --standin --data-dir $DATA \
    --out test_oligo --chroms chr2,chr19
python3 verify_bam.py --bam test_oligo/SIM_OLIGO_HG38.bam --data-dir $DATA --standin
```

3. Lancer la simulation complète sur le vrai hg38 (chr1–22 et chrX par défaut). Le script vérifie d'abord que la base de référence de chaque point chaud est celle attendue en hg38.

```
python3 simulate_glioma_hg38.py --profile gbm --ref $REF --data-dir $DATA \
    --out sim_gbm --purity 0.8 --seed 2026
```

4. Vérifier le BAM. Avec --standin, donner la même --seed que la simulation.

```
python3 verify_bam.py --bam sim_gbm/SIM_GBM_HG38.bam --data-dir $DATA --ref $REF
```

Options principales

Option	Défaut	Effet
`--profile`	—	gbm, astro ou oligo (obligatoire)
`--ref / --standin`	—	FASTA hg38 réelle, ou séquence synthétique de test (l'un des deux)
`--data-dir`	—	Dossier data/ de Rapid-CNS2_nf (obligatoire)
`--out / --sample-id`	sim_glioma / SIM_<PROFIL>_HG38	Dossier de sortie et nom de l'échantillon
`--purity`	0,80	Pureté tumorale ; VAF hétérozygote = pureté/2
`--ontarget-depth`	8	Profondeur par région du panel (couverture effective ~2,5 fois plus haute)
`--offtarget-depth`	2	Profondeur hors panel (sert à l'appel des CNV)
`--mgmt-depth-factor`	4	Profondeur dédiée au promoteur MGMT = facteur × on-target
`--cdkn2a-del`	non	astro : ajoute la délétion homozygote CDKN2A/B (grade 4)
`--cnv-jitter-sd`	0	Bruit multiplicatif sur les rapports de copies
`--chroms`	chr1–22, chrX	Restreint les chromosomes (tests rapides)
`--seed`	2026	Graine : même graine et mêmes options, même BAM
`--mnp-targets / --mnp-depth / --mnp-window`	— / 6 / 150	Profil de méthylation réel aux sondes HM450, profondeur et demi-fenêtre dédiées
`--skip-ref-check`	non	Saute le contrôle des bases de référence

Fichiers produits

- <out>/<id>.bam et <id>.bam.bai : BAM trié et indexé, tags MM/ML, NM, RG ; l'en-tête signale qu'il s'agit de données simulées.
- <out>/truth_events.tsv : vérité terrain, une ligne par événement (colonnes event, type, chrom, start, end, value).

```
event                          type        chrom  start      end        value
chr7_copy_ratio                copy_ratio  chr7   0          159345973  1.5
chr10_copy_ratio               copy_ratio  chr10  0          133797422  0.5
EGFR_high_level_amplification  copy_ratio  chr7   55014032   55207338   9.0
MGMT_promoter_methylated       methylation chr10  129466536  129467536  0.88
TERT_C228T                     snv         chr5   1295112    1295113    G>A;vaf=0.800
```

Points à contrôler

- Taux de mésappariement proche de 2 % (au-delà de 10 % avec --standin : mauvaise --seed).
- Rapports de copies mesurés proches des valeurs « expected » (ex. 0,50 pour une perte d'une copie).
- VAF des points chauds proche de pureté/2, à la variance d'échantillonnage près (profondeur ~17–35×).
- Couverture MGMT au-dessus du seuil choisi pour Rapid-CNS2 (10× dans les lanceurs) et méthylation MGMT ~0,88.

4.2 Classer la méthylation d'un échantillon

Objectif : obtenir la classe de méthylation (Capper et al.) qu'attribuerait Rapid-CNS2, à partir d'un BAM simulé portant le profil réel d'un échantillon de la classe visée. `[simulation/]`

Étapes

1. Procédure automatique (quatre étapes : export du profil représentatif, simulation avec transplantation, bedMethyl, classification).

```
./run_full_mnp_classification.sh --profile astro --data-dir $DATA --ref $REF \
    --num-trees 20000 --threads 16
```

2. Ou étape par étape, par exemple pour relancer une seule étape ; pour un BAM déjà simulé avec --mnp-targets, seules les deux dernières commandes sont nécessaires :

```
Rscript 01_export_probe_targets.R $DATA probe_targets
python3 simulate_glioma_hg38.py --profile astro --ref $REF --data-dir $DATA \
    --out sim_astro_mnp --mnp-targets probe_targets/probe_targets_astro.tsv
python3 bedmethyl_from_bam.py --bam sim_astro_mnp/SIM_ASTRO_HG38.bam \
    --out sim_astro_mnp/SIM_ASTRO_HG38.bedmethyl.bed
Rscript methylation_classification_local.R -s SIM_ASTRO_HG38 -o classif_astro \
    -i sim_astro_mnp/SIM_ASTRO_HG38.bedmethyl.bed -p $DATA/top_probes_hm450.Rdata \
    -a $DATA/HM450.hg38.manifest.gencode.v22.Rdata \
    -d $DATA/capper_top_100k_betas_binarised.Rdata -t 16 -n 20000
```

3. Nombre d'arbres : 20 000 comme en production ; 2 000 pour un criblage rapide (environ dix fois plus court, moins précis). Mémoire mesurée : 4,4 Go avec 300 arbres ; avec 2 000 arbres, plus de 7 Go (processus tué dans l'environnement de test, qui n'en avait que 7).

Fichiers produits

- classif_<profil>/<id>_calibrated_classification.tsv : nombre de sondes, classe prédite, score initial, score calibré.
- <id>_votes.tsv (fréquences par classe), <id>_rf_details.tsv (recouvrements, score de Brier), <id>_methylation_hg38_HM450.RDS, <id>_probes_for_training.csv.

Points à contrôler

- Classe attendue : GBM_RTK_II (gbm), A_IDH (astro), A_IDH_HG (astro grade 4), O_IDH (oligo).
- « Number of features » : 10 000 au plus (sondes d'entraînement, plafonnées à 10 000 ; 10 000 mesuré avec 6 chromosomes, 6 640 avec chr17 seul) ; « Top_100k_overlap » (fichier _rf_details.tsv) proche du nombre de sondes du fichier probe_targets utilisé.

4.3 Simuler une cohorte

Objectif : produire N BAM d'individus distincts (pureté, amplitude des CNV, grade, profil de méthylation réel), puis éventuellement mesurer le taux de classification correcte. `[simulation/]`

Étapes

1. Exporter les profils réels : un fichier par individu et par classe. n_per_class doit valoir l'effectif du plus grand groupe (33 pour N = 99 à parts égales). Si une classe a moins d'échantillons réels que demandé, ils sont réutilisés et un avertissement s'affiche.

```
Rscript 02_export_cohort_probe_targets.R $DATA cohort_probes 33
```

2. Tester d'abord une petite cohorte sur un chromosome.

```
python3 build_cohort.py --n 9 --standin --data-dir $DATA --out-dir cohort_test \
    --chroms chr17 --jobs 3
```

3. Lancer la cohorte complète. --profile-counts impose des effectifs (somme = N) ; --astro-grade4-fraction fait passer une partie des astrocytomes en grade 4 (délétion CDKN2A/B et profil A_IDH_HG).

```
python3 build_cohort.py --n 99 --ref $REF --data-dir $DATA --out-dir cohort_bams \
    --mnp-targets-dir cohort_probes --jobs 16 --cnv-jitter-sd 0.15 \
    --astro-grade4-fraction 0.3 --classify --classify-jobs 2 --num-trees 2000
```

4. Régler --jobs selon la mémoire : environ 0,8 Go mesuré par simulation de chr17 seul, sans sondes MNP, après correction ; la consommation croît avec la longueur du plus grand chromosome simulé (chr1 est trois fois plus long). Un statut FAILED(rc=-9) dans le manifeste signale un processus tué, le plus souvent

par manque de mémoire : réduire --jobs. Pour --classify-jobs, compter plus de 7 Go par classification à 2 000 arbres.

Fichiers produits

- cohort_bams/INDnnn_<PROFIL>/ : BAM, index et truth_events.tsv de chaque individu ; INDnnn_<PROFIL>.log : journal.
- cohort_bams/cohort_manifest.tsv : individu, profil, graine, pureté, grade 4, classe attendue, fichier de sondes, BAM, statut, durée.
- cohort_classification/cohort_classification_summary.tsv : classe attendue et prédite, score calibré, statut ; taux de classification correcte affiché en fin d'exécution.

Points à contrôler

- Toutes les lignes du manifeste au statut OK.
- Avertissement « no probe file found » absent : sinon les individus concernés n'ont pas de vrai signal de méthylation.

4.4 Utiliser l'interface de simulation (Streamlit)

Objectif : préparer et lancer les procédures 4.1 et 4.3 sans écrire les commandes. `[simulation/]`

Étapes

1. Lancer le serveur sur P3Ultra ; depuis un autre poste, ouvrir un tunnel SSH puis http://localhost:8501.

```
streamlit run simnanocns_gui_en.py
ssh -L 8501:localhost:8501 P3Ultra
```

2. Barre latérale : renseigner le dossier de travail, le dossier des scripts (ce dossier simulation/), l'interpréteur Python, le dossier data/ et la référence. « Save as defaults » enregistre ces valeurs dans ~/.simnanocns_gui_en.json ; « Export/Import (JSON) » échange une configuration.
3. Onglet Single sample ou Cohort : régler les paramètres. La colonne de droite montre la commande exacte, signale les options inconnues des scripts installés et liste les contrôles (fichiers présents, espace disque, cohérence).
4. « Run » lance la commande en arrière-plan ; l'onglet Run affiche la durée, le nombre de BAM produits, le journal et, à la fin, la vérité terrain ou le manifeste et le taux de classification. La simulation continue si l'onglet du navigateur est fermé, mais s'arrête si le serveur Streamlit est arrêté.

Fichiers produits

- <dossier de travail>/gui_logs/<date>_<nom>.log et .params.json : journal et paramètres de chaque lancement.

4.5 Lancer Rapid-CNS2_nf sur un BAM

Objectif : analyser un BAM simulé ou réel avec le pipeline et ouvrir son rapport. `[rapidcns2/]`

Étapes

1. Ouvrir l'interface ; choisir le dossier de cohorte (les BAM de <cohorte>/*.bam et <cohorte>/*/*.bam sont listés) et l'échantillon.

```
python3 rapidcns2_gui_en.py
```

2. Vérifier les chemins (main.nf, dossier de lancement, référence, ANNOVAR, AnnotSV) et les options : profil Nextflow (local,docker ou local,singularity), couverture MGMT minimale, CPU, mémoire, pureté pour SAVANA, MNP-Flex, Human variation, reprise (-resume).
3. Pour des BAM simulés, placer dans le dossier de lancement un fichier custom.config qui ignore les échecs attendus (aucun variant de structure, aucun SNP hétérozygote) ; l'interface l'utilise automatiquement s'il existe. À retirer pour des données réelles.

```
process {
    withName: 'annotSv.*' { errorStrategy = 'ignore' }
    withName: 'savanaCna' { errorStrategy = 'ignore' }
}
```

4. « Run » lance Nextflow ; « Copy command » copie la commande équivalente pour un terminal. La barre de progression compte les étapes terminées dans le fichier trace ; « Stop » arrête proprement Nextflow.
5. « Open report » ouvre le rapport HTML (version full, sinon lite).

Fichiers produits

- <sortie>/report/<id>_Rapid-CNS2_report_full.html (et _lite.html) : rapport final.
- <sortie>/mods/<id>.5mC.bedmethyl, <sortie>/mgmt/mgmt_avg_cov.txt, <sortie>/pipeline_info/*_trace.txt.

Points à contrôler

- Comparer le rapport à truth_events.tsv : classe de méthylation, statut MGMT, CNV (+7/−10, EGFR, CDKN2A/B, 1p/19q), variants (IDH1, TERT, TP53).
- mgmt_avg_cov.txt au-dessus de --minimumMgmtCov, sinon le module MGMT est sauté.

4.6 Préparer des données réelles (FASTQ vers BAM)

Objectif : transformer des FASTQ Nanopore en BAM hg38 trié et indexé, utilisable par Rapid-CNS2.
`[rapidcns2/]`

Étapes

1. Vérifier que les FASTQ portent les tags MM/ML dans leurs en-têtes (produits par samtools fastq -T '*'). Un FASTQ MinKNOW standard n'en a pas : la méthylation (classification, MGMT) sera alors impossible ; repartir des POD5 ou du BAM non aligné.
2. Lancer l'alignement : un fichier ou un dossier de FASTQ (compressés ou non), la référence, le nom de l'échantillon ; -m transmet des options à minimap2.

```
./align_fastq_rapidcns2_en.sh -i fastq_pass/ -r $REF -s GBM01 -o results -t 24
```

Fichiers produits

- results/GBM01.sorted.bam (+ .bai), results/GBM01.qc.txt (flagstat, reads par chromosome, reads avec MM), results/GBM01.merged.fastq.gz si un dossier a été fusionné.

Points à contrôler

- Pas d'avertissement « no MM:Z: tag » ni « contig 'chr7' missing ».
- Dans le fichier QC, des reads porteurs de MM parmi les 2 000 premiers alignements.

4.7 Construire et utiliser la clé USB

Objectif : emporter Rapid-CNS2 sur une clé autonome, utilisable hors ligne sur un autre poste Linux.
`[usb/]`

Étapes

1. Méthode recommandée : make_usb_stick_en.sh fait toutes les étapes ci-dessous en une commande, avec des contrôles de sécurité (jamais un disque système) et une confirmation avant effacement. Les étapes manuelles suivantes restent valables.

```
./make_usb_stick_en.sh --list
./make_usb_stick_en.sh --dry-run /dev/sdX
./make_usb_stick_en.sh --eject /dev/sdX -- --engine apptainer --with-samples
/path/to/cohort_bams
```

2. Formater la clé en ext4 (FAT32 et exFAT sont refusés : liens symboliques, permissions Unix, fichiers de plus de 4 Go). Cette opération efface la clé ; la rebrancher avant le chown.

```
lsblk
sudo umount /dev/sdX1
sudo mkfs.ext4 -m 0 -L RAPIDCNS2 /dev/sdX1
sudo chown $USER: /media/$USER/RAPIDCNS2
```

3. Estimer la place nécessaire sans rien écrire.

```
./prepare_usb_en.sh --dry-run /media/$USER/RAPIDCNS2
```

4. Construire la clé. Les chemins sources (pipeline, référence, ANNOVAR, AnnotSV) se règlent par variables d'environnement (PIPELINE_DIR, REF_FA, ANNOVAR_DIR, ANNOTSV_DIR, GUI_PY). --engine choisit apptainer, docker ou both ; --no-annovar exclut ANNOVAR (non redistribuable) ; --with-samples DIR copie des BAM.

```
./prepare_usb_en.sh --engine apptainer --with-samples /path/to/cohort_bams
/media/$USER/RAPIDCNS2
```

5. Sur le poste hôte : contrôler l'installation, puis lancer l'interface ou la ligne de commande. Le dossier de travail Nextflow est sur le disque de l'hôte (~150 Go par échantillon, modifiable par RCNS_WORKDIR).

```
cd /media/$USER/RAPIDCNS2/rapidcns2
./run.sh --check
./run_gui.sh
./run.sh samples/IND001_GBM/IND001_GBM.bam IND001_GBM
```

Fichiers produits

- rapidcns2/ sur la clé : bin/ (Nextflow), jre/, pipeline/, containers/, refs/, annovar/, annotsv/, gui/, samples/, results/, conf/usb.config, env.sh, run.sh, run_gui.sh, README.txt.

Points à contrôler

- run.sh --check affiche « Everything is in place. » (ANNOVAR apparaît manquant si la clé a été construite avec --no-annovar : pointer alors RCNS_ANNOVARPATH et RCNS_ANNOVARDB vers une installation locale).
- Hors ligne, wf-human-variation est désactivé et le dépôt automatique vers MNP-Flex est impossible.
- conf/usb.config ignore les échecs d'AnnotSV et de SAVANA (BAM simulés) : commenter ces lignes pour des données réelles.

4.8 Régénérer les données produites avant les corrections

Objectif : remplacer les fichiers affectés par les erreurs corrigées. `[simulation/]`

Étapes

1. bedMethyl et classifications : à refaire pour tous les BAM (erreur de décodage du brin –). Les BAM eux-mêmes restent valides.

```
python3 bedmethyl_from_bam.py --bam X.bam --out X.bedmethyl.bed
# puis methylation_classification_local.R (procédure 4.2)
```

2. BAM du profil gbm destinés à Rapid-CNS2 : à resimuler pour obtenir la nouvelle couverture MGMT (18–22×).
3. Les nouveaux BAM ne sont pas identiques octet par octet aux anciens pour une même graine (ordre des tirages de méthylation modifié), mais restent reproductibles.

5. Fonctionnement détaillé

5.1 Simulation d'un BAM (simulate_glioma_hg38.py)

1. Référence : lecture de la FASTA hg38 (noms chr* ou RefSeq) ou séquence synthétique déterministe (GC 42 %, CpG appauvris). Contrôle de la base de référence des points chauds.
2. Modèle tumoral : rapports de copies par chromosome et événements focaux du profil (tableau ci-dessous), éventuellement bruités (--cnv-jitter-sd : facteur N(1, sd) borné à [0,5 ; 1,5]).

3. Plan de couverture, par chromosome : segments hors panel (profondeur off-target × rapport local), régions du panel (on-target), cibles dédiées (MGMT, ±300 pb autour des points chauds et du site de délétion), fond supplémentaire dans les fenêtres amplifiées, fenêtres de ±150 pb autour des sondes HM450.
4. Tirage des reads : pour chaque cible, nombre de reads ~ Poisson(profondeur × (largeur + longueur moyenne) / longueur moyenne) ; longueur log-normale (moyenne ~3,1 kb, bornée à 300–40 000 pb ; reads courts de 250–2 000 pb pour les sondes) ; début uniforme pour que chaque read chevauche la cible. Les fragments contenant N ou un code ambigu sont écartés.
5. Allèle tumoral : chaque read couvrant un point chaud porte l'allèle muté avec une probabilité égale à la VAF ; la délétion de 1 pb (ATRX ou CIC) est appliquée avec une probabilité pureté/2 si elle est à plus de 20 pb des extrémités du read.
6. Brin tiré au hasard (50 %). Erreurs de séquençage : 4 % des bases (50 % substitutions, 25 % délétions, 25 % insertions) ; CIGAR et NM reconstruits ; MAPQ 60 ; qualité Q20.
7. Méthylation : pour chaque CpG du read (orientation d'origine), probabilité de méthylation selon le modèle ci-dessous, appel tiré au hasard, valeur ML de 200–255 si méthylé, 0–55 sinon ; tag MM « C+m » avec les écarts entre C de CpG.
8. Écriture : reads triés par position, BAM indexé, en-tête avec le modèle de basecalling dorado et un commentaire « SIMULATED DATA » ; puis truth_events.tsv.

5.2 Profils tumoraux simulés

Altération	gbm	astro	oligo
Chromosome 7 / 10	gain 1,5 / perte 0,5	diploïde	diploïde
1p / 19q	—	—	perte d'une copie (0,5)
EGFR	amplification (×9)	—	—
CDKN2A/B	délétion homozygote (0,02, ±50 kb)	option grade 4	—
IDH1 R132H	sauvage	pureté/2	pureté/2
TERT C228T	0,80	sauvage	pureté/2
TP53 R175H	—	pureté/2	sauvage
Délétion de 1 pb	—	ATRX	CIC
Promoteur MGMT	méthylé (0,88)	méthylé (0,88)	méthylé (0,88)
Fond de méthylation	0,47	0,64 (G-CIMP)	0,64 (G-CIMP)
Classe de méthylation attendue	GBM_RTK_II	A_IDH (A_IDH_HG en grade 4)	O_IDH

5.3 Modèle de méthylation

Site	Probabilité de méthylation
CpG de fond	0,04 ou 0,90 selon un hachage déterministe de la position (même valeur pour tous les reads) ; part des sites à 0,04 : 50 % (gbm) ou 30 % (astro, oligo)
Promoteur MGMT	0,88
Sonde HM450 transplantée	0,92 si l'échantillon réel est méthylé, 0,04 sinon
CpG mal aligné (erreur, insertion)	0,50

5.4 bedMethyl (bedmethyl_from_bam.py)

1. Lecture de chaque read primaire aligné portant un tag MM ; décodage des appels par pysam (positions dans la séquence stockée).
2. Brin + : l'appel est sur le C ; brin – : sur le G opposé. L'appel est ramené à la position du C du CpG sur le brin + ; il est ignoré si le C et le G ne sont pas alignés sur deux positions consécutives de la référence.
3. Agrégation par site : couverture et nombre d'appels méthylés (ML ≥ 128) ; écriture des 18 colonnes modkit, brin « . » (équivalent de --combine-strands).

5.5 Classification (methylation_classification_local.R)

1. Lecture des colonnes 1, 2, 3, 6, 10 et 11 du bedMethyl ; retrait de chrX et chrY ; méthylation = pourcentage / 100.
2. Recouvrement avec le manifeste HM450 hg38 : chaque sonde couverte reçoit la méthylation du site.
3. Données d'entraînement (Capper et al., binarisées) restreintes aux sondes couvertes ; conservation des 10 000 plus variables.
4. Forêt aléatoire probabiliste (ranger), fractions d'échantillonnage équilibrées entre classes ; puis une régression logistique par classe pour recalibrer les scores.
5. Échantillon binarisé au seuil 0,5, classe prédite = probabilité maximale ; écriture du score initial et du score calibré.

5.6 Vérification (verify_bam.py)

1. En-tête et présence des tags MM.
2. Couverture moyenne du panel par chromosome (indicative seulement : les reads longs débordent sur les régions voisines).
3. Rapports de copies des grands événements : densité de reads hors panel par Mb rapportée à la médiane de chromosomes neutres simulés.
4. Événements focaux (EGFR, CDKN2A/B) : couverture brute rapportée au panel d'un chromosome neutre ; c'est le sens du signal qui compte.
5. Points chauds : comptes A/C/G/T et VAF ; MGMT : profondeur et méthylation par brin ; témoin : méthylation de 40 régions du panel hors MGMT.
6. Cohérence avec la référence : taux de mésappariement sur 200 reads (≈ 2 % attendus).

5.7 Cohorte (build_cohort.py)

1. Répartition des N individus entre profils (parts égales, reste donné à gbm puis astro, ou effectifs imposés), ordre mélangé par la graine de base.
2. Pour l'individu i : graine = graine de base + i ; pureté tirée uniformément dans --purity-range ; grade 4 tiré avec la probabilité --astro-grade4-fraction (astro seulement) ; fichier de sondes de rang propre à son profil.
3. Simulations indépendantes en parallèle (--jobs), manifeste ; puis classifications en parallèle (--classify-jobs) et taux de classification correcte.

6. Description des scripts et de leurs fonctions

Pour chaque script : son rôle, puis ses fonctions principales. Les fonctions internes d'interface (construction des widgets, rappels) sont regroupées.

6.1 simulation/simulate_glioma_hg38.py

Génère un BAM Nanopore aligné sur hg38 (panel NPHD, échantillonnage adaptatif) pour les profils gbm, astro et oligo, avec variants, altérations du nombre de copies et méthylation (tags MM/ML), et écrit la vérité terrain dans truth_events.tsv.

Fonction	Rôle
`RealRef`	Lit une FASTA hg38 réelle ; accepte les contigs chr1… ou RefSeq (NC_000001.11…).
`StandInRef`	Séquence synthétique déterministe ; peut forcer des bases (points chauds, CG des sondes HM450).
`read_bed`	Lit un fichier BED (chrom, début, fin, nom).
`build_model`	Charge panel, région MGMT et gènes ; construit les événements du profil.
`frameshift_site`	Site de la délétion de 1 pb : milieu du plus grand exon codant du plus long transcrit NM_.
`ratio_at`	Rapport de copies à une position (événement focal prioritaire).
`add_errors`	Ajoute les erreurs de séquençage et renvoie la position de référence de chaque base.
`cigar_from_rp`	Reconstruit le CIGAR à partir des positions de référence.
`methylation_tags`	Calcule les tags MM/ML conformes à la spécification SAM.
`main`	Options, génération par chromosome, écriture du BAM et de la vérité terrain.

6.2 simulation/verify_bam.py

Vérifie un BAM simulé indépendamment du générateur et compare chaque mesure à truth_events.tsv.

Fonction	Rôle
`density`	Reads hors panel par Mb : estimateur du rapport de copies.
`mean_cov`	Couverture moyenne échantillonnée sur une fenêtre.
`counts`	Comptes A/C/G/T à une position.
`(corps du script)`	Sections de contrôle successives (voir 5.6).

6.3 simulation/build_cohort.py

Simule une cohorte de N individus en parallèle, puis peut classer chaque BAM.

Fonction	Rôle
`parse_profile_counts`	Lit des effectifs imposés et vérifie leur somme.
`build_assignment`	Répartit les individus entre profils, ordre mélangé reproductible.
`run_one`	Simule un individu et renvoie une ligne du manifeste.
`classify_one`	bedMethyl puis classification R ; lit la classe prédite.
`main`	Contrôle des options, génération, manifeste, classification, résumé.

6.4 simulation/bedmethyl_from_bam.py

Produit un bedMethyl au format modkit à partir des tags MM/ML (équivalent de modkit pileup --cpg --combine-strands).

Fonction	Rôle
(script linéaire)	Décodage, report sur le C du brin +, agrégation par site, écriture (voir 5.4).

6.5 simulation/methylation_classification_local.R

Version locale de la classification de Rapid-CNS2 (voir 5.5).

Fonction	Rôle
Lecture et appariement	bedMethyl, retrait chrX/chrY, recouvrement avec le manifeste HM450.
Entraînement	10 000 sondes les plus variables ; ranger équilibré par classe.
Recalibration	Régression logistique une classe contre toutes.
Prédiction	Binarisation, classe prédite, scores et votes.

6.6 simulation/01_export_probe_targets.R / 02_export_cohort_probe_targets.R

Exportent des profils de méthylation réels aux positions hg38 des sondes HM450 : un représentant par classe (01) ou N échantillons distincts par classe (02).

Fonction	Rôle
(scripts linéaires)	Intersection top_probes ∩ manifeste autosomique ∩ betas ; écriture des fichiers probe_targets.

6.7 simulation/run_full_mnp_classification.sh

Chaîne complète pour un profil (procédure 4.2).

Fonction	Rôle
(script linéaire)	Validation des options, quatre étapes, affichage du résultat.

6.8 simulation/simnanocns_gui_en.py

Interface Streamlit qui construit, vérifie et lance les commandes de simulation (procédure 4.4).

Fonction	Rôle
build_single_cmds / build_cohort_cmds	Construisent les commandes.
equal_counts / group_counts / per_class	Effectifs par groupe (méthode du plus fort reste).
preflight / cohort_errors	Contrôles avant lancement.
parse_help_flags / unknown_flags	Options inconnues d'une ancienne version des scripts.
launch / stop_job / tab_run	Lancement en arrière-plan, arrêt, suivi.

Fonction	Rôle
`Autres (sidebar, tab_*, show_*)`	Interface, configuration.

6.9 `rapidcns2/rapidcns2_gui_en.py`

Interface Tkinter de lancement de Rapid-CNS2_nf (procédure 4.5) ; valeurs par défaut modifiables par variables RCNS_*.

Fonction	Rôle
`find_bams`	Liste les BAM d'un dossier de cohorte.
`build_command`	Construit la commande nextflow run.
`count_trace / task_name`	Comptent les étapes terminées dans le fichier trace.
`run / _reader / _poll_log / stop`	Lancement, lecture du journal, arrêt.
`_poll_progress / _finish_progress`	Progression et état final.
`open_report`	Ouvre le rapport HTML.

6.10 `rapidcns2/align_fastq_rapidcns2_en.sh`

FASTQ vers BAM hg38 trié et indexé, avec contrôles (procédure 4.6).

Fonction	Rôle
`usage / log / warn / die`	Aide et messages.
`(étapes 1 à 5)`	Fusion, méthylation, alignement et tri, index, contrôles.

6.11 `usb/make_usb_stick_en.sh`

Crée la clé de bout en bout : contrôles de sécurité, effacement et formatage ext4 après confirmation, montage, appel de prepare_usb_en.sh, contrôle final (procédure 4.7).

Fonction	Rôle
`run`	Exécute une commande, ou l'affiche seulement avec --dry-run.
`part1`	Nom de la première partition (sdb1, nvme0n1p1, loop0p1).
`Contrôles`	Refuse un disque monté hors de /media, /run/media, /mnt ou utilisé comme swap ; exige un disque USB ou amovible.
`Partition et formatage`	wipefs, parted (ou sgdisk) en GPT, mkfs.ext4 -m 0 -L RAPIDCNS2.
`Montage et fin`	udisksctl ou mount, chown à l'utilisateur, prepare_usb_en.sh, run.sh --check, éjection facultative.

6.12 `usb/prepare_usb_en.sh`

Construit la clé USB autonome et écrit ses lanceurs (procédure 4.7).

Fonction	Rôle
`sz / human / sif_name`	Taille, affichage, nom d'image dans le cache Nextflow.

Fonction	Rôle
`rcns_check_engine / rcns_load_docker (env.sh)`	Vérifient le moteur de conteneurs et chargent les images Docker.
`run.sh --check`	Contrôle de l'installation sur l'hôte.

7. Erreurs trouvées et corrections

Classées par importance. « Vérification » indique comment la correction a été contrôlée.

Fichier	Problème	Correction	Vérification
`bedmethyl_from_bam.py`	Reads du brin – mal décodés : le tag MM compte les C du read d'origine, le script comptait ceux de la séquence stockée. 95 % des appels hors CpG.	Décodage par pysam ; appels ramenés au C du brin + ; CpG non contigus ignorés ; brin « . ».	Appels du brin – sur un CpG : 5 % → 76 % (mesuré avant le filtre de contiguïté) ; MGMT 0,89 (attendu 0,88).
`simulate_glioma_hg38.py`	Positions de chaque base gardées pour tous les reads d'un chromosome : individu tué par manque de mémoire.	Tags calculés à la création du read ; ML en uint8.	chr17 : 2,1 → 0,77 Go ; cohorte test 3/3 au lieu de 2/3.
`simulate_glioma_hg38.py`	Profondeur MGMT figée : ~4–7× en gbm, sous le seuil de 10× des lanceurs.	Option --mgmt-depth-factor (4 par défaut).	MGMT 17,9× et 22,5×.
`simnanocns_gui_en.py`	Graine non transmise à verify_bam.py en mode stand-in (~74 % de mésappariements) ; samtools exigé à tort.	--seed transmis ; contrôle samtools retiré ; message .fai clarifié ; journal fermé.	Commande générée et contrôles testés.
`verify_bam.py`	Fond attendu codé à 0,47 ; référence sur chr1 non neutre pour oligo ; événements non simulés affichés à 0,00 ; samtools obligatoire.	Valeurs lues dans truth_events.tsv ; chromosome neutre automatique ; « not simulated » ; pysam.bedcov.	19q mesuré 0,50 (attendu 0,50).
`simulate_glioma_hg38.py, build_cohort.py`	Arguments non validés : chromosome inconnu → plantage tardif ; pureté > 1 acceptée.	Contrôle de --chroms, --purity, --purity-range, --n, --jobs, fraction de grade 4.	Messages d'erreur testés.
`methylation_classification_local.R`	Égalité de probabilités → plantage de la recalibration ; fichier d'entraînement par défaut faux ; options obligatoires non contrôlées.	Classe par max.col ; défaut corrigé ; contrôles ; scores nommés.	Exécuté sur un BAM gbm simulé : GBM_RTK_II prédit.
`methylation_classification_local.R`	Mémoire : le bedMethyl entier (18 colonnes) et ses copies restaient en mémoire pendant l'entraînement ; processus tué à 6 Go avant même la forêt aléatoire.	Lecture des 6 colonnes utiles (fread) ; tables libérées avant de charger les données d'entraînement.	Classification menée à son terme (300 arbres, 4,4 Go).
`01_ / 02_export_*.R`	Plantage si une classe est absente ; conversion numérique fragile ; arguments non contrôlés.	Classe absente ignorée avec avertissement ; as.matrix ; contrôles.	Exécutés : 100 000 sondes par classe ; contrôle d'argument testé.

Fichier	Problème	Correction	Vérification
`align_fastq_rapidcns2_en.sh`	grep -q avec pipefail : faux avertissement possible.	Liste des contigs capturée avant le test.	shellcheck sans avertissement.
`prepare_usb_en.sh`	Interface introuvable après regroupement ; binaire Nextflow « -dist » absent des anciennes versions ; --engine none accepté.	Recherche dans ../rapidcns2/ ; essai -dist puis -all ; moteurs limités ; contrôles d'arguments.	Clé construite en bac à sable + run.sh --check.
`rapidcns2_gui_en.py`	Noms d'étapes préfixés mal lus ; rapport « full » seul recherché.	Nom normalisé ; repli sur « lite ».	Expression régulière testée.
`rapidcns2_gui_en.py`, `prepare_usb_en.sh` (`env.sh`)	Avec Nextflow 26.04 (syntaxe stricte par défaut), Rapid-CNS2_nf 3.0 ne démarre plus : « Config parsing failed », import dans nextflow.config, instructions hors du workflow dans main.nf (10 erreurs).	NXF_SYNTAX_PARSER=v1 réglé par l'interface et par env.sh, sauf valeur déjà fixée ; le pipeline n'est pas modifié.	Erreur reproduite avec Nextflow 26.04.6 ; pipeline démarré avec v1.
`rapidcns2_gui_en.py`	Hors clé USB, --containerBindPaths n'était pas passé : AnnotSV puis ANNOVAR, lancés dans Docker, ne voyaient pas leurs dossiers (chemins transmis comme du texte, donc non montés). Run arrêté à annotSvSeverus.	Dossiers AnnotSV, ANNOVAR et humandb montés automatiquement (sans doublons ni sous-dossiers), complétés par RCNS_BINDPATHS.	Options Docker vérifiées avec Nextflow 26.04.6 : -v pour chaque dossier.
`rapidcns2_gui_en.py`	Sur BAM simulé, Severus et Sniffles ne trouvent aucun variant de structure : AnnotSV sort sans écrire sa table et Nextflow arrête tout le pipeline (« Missing output file »). Il fallait écrire un custom.config à la main.	Case « Simulated BAM », cochée d'office si l'en-tête du BAM contient « SIMULATED DATA » ; elle écrit et ajoute .rapidcns2_simulated.config (AnnotSV et SAVANA CNA ignorés, sorties non utilisées par le rapport ou avec repli).	Détection testée (BAM simulé, BAM ordinaire, fichier absent) ; règle vérifiée avec Nextflow 26.04.6 sur annotSvSeverus, annotSvSniffles et savanaCna.
`build_cohort.py`, `run_full_mnp_classification.sh`	Faux avertissement pour un groupe vide ; ligne vide non gérée ; profil non validé.	Avertissement limité ; lecture robuste ; profil vérifié.	Cohorte de 3 individus générée.

7.1 Points vérifiés et laissés tels quels

- --savanaCnaArgs '--overrule_cellularity 0.8' : Nextflow accepte une valeur commençant par un tiret si elle contient une espace (fonction isValue du lanceur Nextflow).
- Tous les paramètres passés à Rapid-CNS2_nf existent dans la version actuelle du dépôt, et les noms d'étapes suivis correspondent aux alias de main.nf.
- Les coordonnées hg38 des points chauds (TERT C228T, IDH1 R132H, TP53 R175H) sont correctes.
- L'algorithme de classification reste identique à celui de Rapid-CNS2 : seules des corrections de robustesse ont été faites.

8. Explication des choix

8.1 Regroupement par usage

La simulation et la classification restent dans le même dossier, car build_cohort.py, run_full_mnp_classification.sh et l'interface Streamlit appellent ces scripts par le chemin de leur propre dossier : les séparer aurait cassé ces appels. L'interface Rapid-CNS2 et l'alignement des FASTQ servent au lancement du pipeline ; la clé USB ne sert qu'au déploiement.

8.2 Imiter modkit plutôt que l'inventer

Rapid-CNS2_nf appelle modkit pileup avec --cpg --combine-strands. Reproduire ce comportement garantit que la classification reçoit le même type d'entrée qu'avec des données réelles. Le brin « . » devient « * » dans R (conversion vérifiée dans le code de GenomicRanges, makeGRangesFromDataFrame) et se superpose aux sondes HM450 quel que soit leur brin. Le « + » utilisé auparavant faisait perdre la moitié des sondes : le manifeste en compte 49 806 sur le brin « − » sur 100 000.

8.3 Utiliser pysam pour décoder MM/ML

Le décodage manuel était à l'origine de l'erreur. pysam implémente la spécification SAM ; son comportement a été vérifié sur les BAM simulés (C pour le brin +, G opposé pour le brin –).

8.4 Mémoire contre reproductibilité exacte

Calculer les tags à la création du read change l'ordre des tirages de méthylation : les BAM restent reproductibles pour une graine donnée mais ne sont plus identiques octet par octet aux anciens. Ce compromis a été retenu car le gain (×2,7) rend possibles de nombreuses simulations parallèles sur 128 Go.

8.5 Profondeur MGMT paramétrable

Le seuil --minimumMgmtCov vaut 5 par défaut dans Rapid-CNS2 mais 10 dans l'interface et la clé USB. Avec l'ancienne valeur figée, le module MGMT risquait d'être sauté pour le gbm. Un facteur 4 donne 18–22× ; il reste modifiable pour tester le pipeline à faible couverture.

8.6 Valider les arguments au départ

Une simulation complète dure longtemps : une faute de frappe (chromosome inconnu, pureté hors bornes) doit arrêter le script immédiatement avec un message clair plutôt qu'en cours de route.

8.7 Ne pas modifier l'algorithme de classification

Le but est de tester Rapid-CNS2 : la classification locale doit rester fidèle à celle du pipeline. Seuls les cas d'échec ont été corrigés.

9. Tests réalisés

Tests sur séquence de remplacement (stand-in), chromosomes restreints ; scripts R exécutés sur les données réelles de Rapid-CNS2 (R 4.3, paquets Ubuntu). Valeurs mesurées par verify_bam.py, par lecture directe des BAM ou dans les sorties de la classification.

Contrôle	Mesuré	Attendu ou référence
Méthylation promoteur MGMT (gbm)	0,89	0,88
Fond de méthylation (gbm)	0,47	0,47
Sondes HM450 transplantées retrouvées	296 / 300	300
Méthylation des sondes cible 0 / cible 1	0,03 / 0,91	0,04 / 0,92
Rapport de copies 19q (oligo)	0,50	0,50

Contrôle	Mesuré	Attendu ou référence
Fréquence de la délétion CIC (n = 9 reads)	0,33	0,40
VAF TP53 R175H (astro, pureté 0,724)	0,38	0,36
Taux de mésappariement	2,0 %	~2 %
Couverture MGMT (gbm, deux graines)	17,9× / 22,5×	≥ 10×
Mémoire de pointe, chr17	0,77 Go	2,1 Go avant correction
Cohorte test (3 individus, chr17, 3 jobs)	3/3 en 1,6 min	2/3 en 2,7 min avant correction
Export 01 (profils représentatifs, données Capper)	100 000 sondes/classe en 23 s	sans erreur
Export 02 (2 profils par classe)	8 fichiers + manifeste	sans erreur
Classification R, gbm simulé (6 chromosomes, 300 arbres)	GBM_RTK_II, score 0,84	GBM_RTK_II
Cohorte classée (3 individus, chr17, profils Capper réels)	3/3 correctes	GBM_RTK_II, A_IDH, O_IDH
Mémoire de la classification (300 arbres)	4,4 Go	tuée à 6 Go avant correction
make_usb_stick_en.sh sur un disque virtuel de 6 Go (partition, ext4, montage, contenu, contrôle)	OK, puis --keep-format --eject OK	OK
make_usb_stick_en.sh avec le dossier de montage d'une clé déjà formatée	partition retrouvée, rien d'effacé, clé complétée	OK
make_usb_stick_en.sh sur le disque système	refusé, même avec --allow-non-removable	refusé
Construction de clé USB + run.sh --check	OK	OK (ANNOVAR absent avec --no-annovar)
Analyse statique (pyflakes, shellcheck)	aucune erreur	aucune erreur

10. Limites et recommandations

- La classification n'a été testée qu'avec 300 arbres (2 000 dépassent la mémoire de l'environnement de test) et sur 6 chromosomes : à valider sur P3Ultra avec run_full_mnp_classification.sh, avec 2 000 puis 20 000 arbres, pour chaque profil.
- La recalibration émet des avertissements de non-convergence de la régression logistique (séparation parfaite des scores) : ils viennent de la méthode de Rapid-CNS2 et n'affectent pas la classe prédite, mais le score calibré est alors proche de 1.
- Les tests ont utilisé la séquence de remplacement : un essai avec le vrai hg38 (--ref) puis un passage complet dans Rapid-CNS2_nf restent nécessaires.
- Le génome entier n'a pas été simulé ici. Mesure partielle avec sondes MNP : 15,5 min pour 6 chromosomes (chr1, 2, 7, 10, 17, 19, environ un quart du génome) puis 5,6 min pour le bedMethyl ; faire un premier essai avec peu de jobs avant une grande cohorte.
- La profondeur aux points chauds reste modeste (~17–35×) : la VAF mesurée varie d'un échantillon à l'autre (0,12 observé pour IDH1 avec 17 reads, attendu 0,40). C'est du bruit d'échantillonnage ; augmenter --ontarget-depth si une VAF stable est nécessaire.
- Le contournement NXF_SYNTAX_PARSER=v1 dépend de l'ancien analyseur de Nextflow, appelé à disparaître : Rapid-CNS2_nf devra être migré vers la syntaxe stricte (en amont), ou Nextflow fixé à une version qui garde v1 (NXF_VER).

- make_usb_stick_en.sh a été testé sur un disque virtuel (loop), sans udisksctl : à essayer sur une vraie clé USB, d'abord avec --dry-run.
- Les interfaces graphiques (Streamlit, Tkinter) ont été vérifiées sur leur logique, pas sur leur affichage.